

Parte 4/5	video	02:44
Lab 3 - Executando Carga e Sincronização Incremental - Parte 5/5	video	03:11
Lab 3 - Visualizando o Que o Airbyte Realmente Executa	video	04:51
Lab 3 - Conclusão	video	02:01
Arquivos Usados Neste Capitulo	pdf	
Bibliografia, Referências e Links Úteis	pdf	

10. SQL Avançado Para Transformações de Dados

Introdução	pdf	
A Importância da Linguagem SQL na Engenharia de Dados	video	03:12
Uso de Subconsultas e CTEs na Linguagem SQL		



Funções de Janela e Análise com Linguagem SQL

As funções de janela, também conhecidas como funções analíticas, são extremamente úteis em SQL para realizar cálculos complexos em um conjunto de linhas relacionadas de uma tabela.

Elas permitem que você execute cálculos sem agrupar linhas em um único resultado de saída, mantendo a granularidade original dos dados enquanto aplica operações como sumarização, médias móveis, rankings e mais.

Conceitos Básicos

Antes de apresentar os exemplos, aqui estão alguns conceitos fundamentais sobre funções de janela em SQL:

- OVER():** A cláusula que define a "janela" sobre a qual a função opera. Dentro de **OVER()**, você pode especificar particionamento, ordenação e o intervalo de linhas para cálculo.
- PARTITION BY:** Usado para dividir os dados em partições distintas, similar a um agrupamento, mas sem reduzir o número de linhas retornadas.
- ORDER BY:** Usado dentro de **OVER()** para especificar a ordem das linhas para a função de janela.
- ROWS/RANGE:** Define o subconjunto de linhas em cada partição ao qual a função é aplicada.

Exemplos de Funções de Janela

1. ROW_NUMBER():

Atribui um número único para cada linha, começando com 1 para a primeira linha em cada partição.

```
1 SELECT
2     nome,
3     departamento,
4     salario,
5     ROW_NUMBER() OVER (PARTITION BY departamento ORDER BY salario) AS row_num
6 FROM empregados;
```

2. SUM() e AVG():

Calcula a soma ou média para um conjunto definido de linhas.

```
1 SELECT
2     data,
3     valor,
4     SUM(valor) OVER (PARTITION BY data) AS soma_diaria,
5     AVG(valor) OVER (PARTITION BY data) AS media_diaria
6 FROM vendas;
```

3. LEAD() e LAG():

Acessa dados da linha seguinte ou anterior sem realizar um auto-join. Útil para comparar valores em sequências.

```
1 SELECT
2     data,
3     valor,
4     LAG(valor, 1) OVER (ORDER BY data) AS valor_anterior,
5     LEAD(valor, 1) OVER (ORDER BY data) AS valor_seguinte
6 FROM vendas;
```

4. RANK() e DENSE_RANK():

Fornece um ranking para cada linha dentro de uma partição. RANK() deixa lacunas nos rankings em caso de empates, enquanto **DENSE_RANK()** não.

```
1 SELECT
2     nome,
3     departamento,
4     salario,
5     RANK() OVER (PARTITION BY departamento ORDER BY salario) AS rank
6 FROM empregados;
```

5. NTILE():

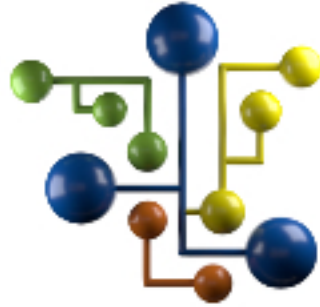
Divide as linhas ordenadas em um número definido de aproximadamente grupos de igual tamanho.

```
1 SELECT
2     nome,
3     salario,
4     NTILE(4) OVER (ORDER BY salario DESC) AS quartil
5 FROM empregados;
```

Usos Avançados

Funções de janela podem ser particularmente poderosas quando combinadas com outras operações **SQL**, como **WHERE** e **JOIN**, e podem ser usadas para resolver problemas complexos como cálculos cumulativos, médias móveis, e mais.

Ao utilizar essas funções, você pode realizar análises detalhadas e complexas diretamente no banco de dados sem a necessidade de processamento adicional em um aplicativo ou camada de análise, otimizando a performance e a eficiência de suas operações de dados.



Equipe DSA

Continue trilhando uma excelente jornada de aprendizagem.

Suporte

Suporte

Suporte

Suporte

Suporte

Suporte

Suporte

Suporte

Suporte

Suporte